KUBA PLATFORM — MOBILE ARCHITECTURE GUIDE

دليل مطور الموبايل (Flutter) لواجهات منصة Kuba

مرجع شامل وتفصيلي يوضح بالتحديد رحلة المستخدم ومسار البيانات (Data Flow) من أول فتح التطبيق، شاشات التعريف، المصادقة، التصفح والبحث الذكي، تفاصيل العقارات والغرف واليخوت وعربات الجولف، وفحص التوفر والحجز، وحتى إدارة الحجوزات والمفضلة والملف الشخصي.

المنصة والتقنية المستهدفة
Flutter (iOS & Android) — REST API

إصدار الباك إند
Kuba Node.js v1.0.0 (Express 5 + MongoDB)

نطاق التغطية
100% لجميع الـ Endpoints (Client & Admin)

تاريخ المراجعة
أكتوبر 2026

📑 فهرس الدليل الشامل

1. التجهيزات الأساسية والـ Headers وقواعد السيرفر
2. خريطة رحلة التطبيق (يروح فين ويجي منين)
3. رحلة المستخدم خطوة بخطوة (Step-by-Step Flow)
4. محرك البحث والفلترة وحساب التوفر الذكي
5. نظام الحجوزات الشامل (فنادق، يخوت، جولف كار، شاليهات)
6. القاموس الفني الكامل لجميع الـ Endpoints (45+ APIs)
7. نصائح وأكواد مساعدة جاهزة لـ Flutter (Dio & Models)

1. التجهيزات الأساسية وقواعد الاتصال بالـ Backend

🌐 روابط السيرفر (Base URLs) حسب بيئة التشغيل

يجب على مطور الفلاتر تهيئة الـ Base URL في ملف الإعدادات وفقاً للجهاز أو المحاكي:

البيئة / الجهاز	الرابط (Base URL)	ملاحظات
Android Emulator	`http://10.0.2.2:3000/api/v1`	الـ IP الخاص بـ Loopback في محاكي أندرويد الافتراضي
iOS Simulator / Desktop	`http://localhost:3000/api/v1`	يعمل مباشرة على شبكة الماك
جهاز حقيقي (Physical Device)	`http://192.168.x.x:3000/api/v1`	استبدل بـ IP جهاز الكمبيوتر على نفس شبكة الواي فاي
Production Server	`https://api.kuba-app.com/api/v1`	سيرفر الإنتاج الفعلي (HTTPS)

🛡️ الهيدرز الإلزامية (Standard Request Headers)

يجب إرسال هذه الـ Headers في كل طلب عبر الـ Interceptor الخاص بـ Dio:

```
{
  "Content-Type": "application/json",
  "Accept-Language": "ar", // يدعم "ar" أو "en" (لتحديد لغة الخطأ والنجاح مترجمة جاهزة للعرض)
  "Authorization": "Bearer <TOKEN>" // يُرسل فقط في الـ APIs المحمية بعد تسجيل الدخول
}
```

📦 الهيكل الموحد لردود السيرفر (Unified JSON Envelope)

جميع ردود السيرفر تتبع نمطاً ثابتاً، مما يسهل كتابة Generic Model في Flutter:

1. الرد الناجح (Success Response - HTTP 200 / 201):

```
{
  "status": "success",
  "message": "تمت العملية بنجاح", // اختياري
```

```
  "data": { ... },                // يحتوي الكائن أو القائمة
  "results": 15,                  // عدد العناصر عند جلب مصفوفات
  "total": 120,                   // إجمالي العناصر
  "page": 1,                      // رقم الصفحة الحالية
  "totalPages": 8,                // إجمالي الصفحات
  "token": "eyJhbGciOi ... "      // عند تسجيل الدخول أو إنشاء الحساب
}
```

2. رد الخطأ (Fail / Error Response - HTTP 400 / 401 / 403 / 404 / 500):

```
{
  "status": "fail", // أو "error" 500 في أخطاء السيرفر
  "message": "رسالة مترجمة بلغة الـ//  ","كلمة المرور غير صحيحة Header
  "errors": [ ... ]                  // تفاصيل إضافية في حال validation errors
}
```

⚠️ **ملاحظة هامة للصور:** السيرفر يقوم بحفظ الصور داخل مجلد `/uploads/` . قد يعيد السيرفر رابط الصورة كاملاً مثل `https://api.kuba.com/uploads/ ...` أو مساراً نسبياً مثل `banners/image-123.jpg` . يجب إنشاء دالة مساعدة (Helper) في تطبيق الفلاتر تضيف `http://<BASE_URL>/uploads/` إذا لم يكن الرابط يبدأ بـ `http` .

2. خريطة رحلة التطبيق (يروح فين ويجي منين)

هذا المخطط يمثل الدليل التوجيهي لمطور الفلاتر: كيف يتحرك المستخدم بين الشاشات، وما هي الـ API المسؤولة عن كل خطوة:

1 شاشة البداية (Splash)

فحص وجود التوكن المخزن محلياً.

➡️ إذا وجد توكن: `GET /auth/me` ثم للرئيسية

➡️ إذا لم يوجد: شاشات التعريف أو المدينة

2 شاشات التعريف (Onboarding)

استعراض مزايا التطبيق للزائر لأول مرة.

➡️ `GET /onboarding` ثم الانتقال لاختيار المدينة

3 اختيار المدينة (City Picker)

الساحل الشمالي، الجونة، القاهرة، شرم الشيخ...

➡️ `GET /cities` وحفظ المدينة في الـ State

4 المصادقة أو التصفح كزائر

تسجيل الدخول، إنشاء حساب، أو تصفح كـ Guest.

➡️ `POST /auth/login` أو `/signup` أو `/google`

5 الرئيسية (Home Screen)

بانرات، تصنيفات، مناطق، وعقارات مقترحة.

➡️ السريعة:
`GET /cities/:id/data?position=top`

6 شاشة المنطقة (Region Screen)

عند الضغط على منطقة (مثل هاسيندا، مراسي...).

➡️ `GET /regions/:id/data` لجلب إعلانات المنطقة

7 البحث المتقدم (Search & Filter)

فلترة بالتواريخ، عدد النزلاء، السعر، والخريطة.

➡️ `GET /search` أو `POST /search`

8 تفاصيل العقار (Property Details)

فندق، شاليه، فيلا، يخت، عربة جولف.

➡️ `GET /advertisements/:id` و `/available-rooms`

9 فحص التوفر وتقدير السعر

حاسبة السعر قبل الدفع (ساعات، أيام، ليالي).

➡️ `POST /bookings/calculate-price`

10 إنشاء وتأكيد الحجز

إدخال بيانات الضيوف، طريقة الدفع، وتثبيت الحجز.

➡️ `POST /bookings` (يتطلب Token)

11 قائمة حجوزاتي (My Bookings)

متابعة الحجوزات السابقة والقادمة وإلغاؤها.

➡️ `GET /bookings` g `PATCH /:id/cancel`

12 المفضلة والملف الشخصي

قائمة الرغبات بزر القلب، وتعديل البروفايل.

➡️ `POST /wishlist/toggle` g `GET /auth/me`

3. رحلة المستخدم خطوة بخطوة (Step-by-Step Flow)

شرح مفصل لكل خطوة مع الطلب المطلوب (Request) والرد المتوقع (Response) وما يفعله المطور في تطبيق Flutter:

الخطوة الأولى — مرحلة بدء التطبيق (App Startup & Splash)

شاشة الـ Splash وفحص الجلسة (Auto Login)

عند فتح التطبيق، يقوم تطبيق الفلاتر بالتحقق مما إذا كان هناك رمز JWT Token محفوظ في `flutter_secure_storage` :

السيناريو أ: يوجد توكن محفوظ:

يتم إرسال طلب للتحقق من هوية المستخدم وتحديث بياناته:

`GET` `/auth/me` `يحتاج توكن`

إذا رد السيرفر بـ `200 OK` : احفظ بيانات المستخدم في UserProvider وتوجه مباشرة إلى **الشاشة الرئيسية**.

إذا رد السيرفر بـ `401 Unauthorized` (التوكن منتهي): قم بمسح التوكن من الهاتف وافتح **شاشة تسجيل الدخول**.

السيناريو ب: لا يوجد توكن:

تحقق هل التطبيق يفتح لأول مرة (`isFirstTime = true`)؟ إذا نعم، توجه لشاشة **Onboarding**، وإلا توجه لشاشة **اختيار المدينة** أو **الرئيسية كزائر (Guest)**.

الخطوة الثانية — شاشات التعريف (Onboarding Screens)

عرض شرائح تعريف المنصة

تُعرض للعميل الجديد للتعريف بخدمات Kuba (حجز الشاليهات، الفنادق، اليخوت، وعربات الجولف).

`GET` `/onboarding?active=true` `عام`

الرد المتوقع:

```
{
  "status": "success",
  "results": 3,
  "data": {
    "screens": [
      {
        "_id": "66000001",
        "title": "فخامة الإقامة في الساحل",
```

```
        "description": "احجز أفخم الفلل والشاليهات مباشرة بأفضل الأسعار",
        "image": "http://api.kuba.com/uploads/onboarding/slide1.jpg",
        "order": 1
      }
    ]
  }
}
```

💡 **نصيحة فلاتر:** استخدم `PageView.builder` مع مؤشر صفحات (SmoothPageIndicator)، وعند الضغط على "تخطي" أو إكمال الشرائح، احفظ في `SharedPreferences: setBool('onboarding_seen', true)` وتـ لتـ اذ الـ

الخطوة الثالثة — اختيار المدينة (City Selection Gate)

تحديد وجهة العميل الرئيسية (الساحل، الجونة، القاهرة...)

تطبيق Kuba قائم على المدن والمناطق السياحية الفاخرة. جميع الإعلانات والبانرات ترتبط بالمدينة المختارة.

`GET` `/cities?active=true` `عام`

الرد المتوقع:

```
{
  "status": "success",
  "results": 2,
  "data": {
    "cities": [
      {
        "_id": "66c82301abc123",
        "name": "الساحل الشمالي",
        "slug": "north-coast",
        "image": "http://api.kuba.com/uploads/cities/sahel.jpg",
        "color": "#2D4F2E",
        "regionsCount": 12
      }
    ]
  }
}
```

ما يفعله الفلاتر: يحفظ معرف المدينة `selectedCityId` في الـ Global State (Bloc / Riverpod / Provider) وفي الذاكرة المحلية، وإذا كان المستخدم مسجلاً دخوله يمكن مزامنتها مع حسابه عبر:

`POST` `/auth/select-city` `يحتاج توكن` (Body: `{"cityId": "..."}`)

الخطوة الرابعة — مسار التسجيل والمصادقة (Authentication Lifecycle)

إنشاء حساب، تسجيل الدخول، واستعادة كلمة المرور

يدعم التطبيق 3 مسارات رئيسية للمصادقة:

المسار 1: إنشاء حساب جديد وتأكيد البريد (Sign Up Flow)

1. **إرسال بيانات التسجيل:**

`POST` `/auth/signup`

:Body

```
{
  "name": "أحمد محمد",
  "email": "ahmed@example.com",
  "phone": "+201012345678",
  "password": "Password123@",
  "confirmPassword": "Password123@"
}
```

يعود السيرفر بـ `201 Created` مع رسالة بأن كود / رابط التفعيل أُرسل للبريد.

2. **تأكيد البريد الإلكتروني:**

`GET` `/auth/verify-email/:token` (أو عبر الرابط المرسل).

3. **إعادة إرسال رمز التفعيل إذا لم يصل:**

`POST` `/auth/resend-verification` (Body: `{"email": "..."}`).

المسار 2: تسجيل الدخول (Login Flow)

- **تسجيل الدخول بالإيميل وكلمة المرور:**

`POST` `/auth/login` (Body: `{"email": "...", "password": "..."}`)

الرد: يعود بـ `token` وكائن `data.user`. قم بتخزين الـ `token` فوراً.

- **تسجيل الدخول بحساب جوجل (Google / Firebase Sign-In):**

في الفلاتر يتم استخدام باقة `google_sign_in` للحصول على `idToken`، ثم إرساله للباك إند:

`POST` `/auth/google` (Body: `{"idToken": "firebase_id_token_here"}`)

المسار 3: استعادة كلمة المرور عبر الـ OTP (Forgot Password 3-Steps)

1. **طلب رمز الاستعادة:** `POST` `/auth/forgot-password` (Body: `{"email": "ahmed@example.com"}`) ➡️ يرسل كود OTP من 6 أرقام للإيميل.

2. **التحقق من الـ OTP:** `POST` `/auth/verify-reset-otp` (Body: `{"email": "...", "otp": "481920"}`) ➡️ يعود السيرفر بـ `resetToken` مؤقت.

3. تعيين كلمة السر الجديدة: `PATCH` `/auth/reset-password/:resetToken` (Body: `{"password": "NewPassword123@", "confirmPassword": "NewPassword123@"}`) ➡️ يتم التعيين ويُمنح المستخدم جلسة جديدة.

الخطوة الخامسة — الشاشة الرئيسية (Home Screen)

بناء واجهة الاستكشاف بطلب واحد أو عدة طلبات

تتكون الصفحة الرئيسية من: شريط المدينة المحددة، بانرات العروض العلوية، قائمة التصنيفات الأفقية (فنادق، يخوت، جولف كار، شاليهات)، قائمة المناطق (Regions)، وأحدث الإعلانات المقترحة.

🚀 **الـ Endpoint السحرية الشاملة لصفحة المدينة (All-in-One City Screen API):**

لتفادي إرسال 4 طلبات شبكة متتالية، وفر الباك إند مساراً مجمعاً يعيد كل ما تحتاجه الشاشة في استدعاء واحد:

`GET` `/cities/:id/data?position=top`

هيكل الرد الشامل لصفحة المدينة:

```
{
  "status": "success",
  "data": {
    "city": { "_id": "...", "name": "الساحل الشمالي", "image": "..." },
    "regions": [
      { "_id": "reg1", "name": "سيدي عبد الرحمن", "image": "..." },
      { "_id": "reg2", "name": "مراسي", "image": "..." }
    ],
    "banners": [
      { "id": "b1", "image": "...", "position": "top", "advertisementId": "ad123"
    ],
    "advertisements": [
      {
        "_id": "ad1",
        "title": "فيلا مستقلة صف أول على البحر",
        "price": 15000,
        "pricing": { "finalPrice": 15000, "currency": "EGP", "pricingUnit": "per_n
        "image": "...",
        "type": "villa"
      }
    ]
  }
}
```

البديل (طلبات منفصلة إذا رغبت في عزل الـ Caching):

- البانرات: `GET` `/banners?city={cityId}&position=top`

- التصنيفات: GET `/categories`
- المناطق: GET `/regions?city={cityId}`
- أبرز العقارات: GET `/advertisements?city={cityId}&limit=10`

الخطوة السادسة — شاشة تفاصيل المنطقة (Region View)

استعراض عقارات قرية أو منطقة معينة (مثل هاسيندا أو سيدي حنيش)

عندما يضغط المستخدم على أي بطاقة منطقة من الصفحة الرئيسية، ينتقل لشاشة تفاصيل المنطقة:

GET `/regions/:id/data` عام

يعيد السيرفر: بيانات المنطقة، البانرات الخاصة بها، الشاليهات، وجميع الإعلانات التي تقع داخل نطاق هذه المنطقة الجغرافي.

الخطوة السابعة — محرك البحث المتقدم والفلترة (Search Engine)

البحث بالتواريخ، عدد النزلاء، السعر، والخريطة الجغرافية

محرك بحث Kuba يعتبر من أقوى أجزاء النظام؛ فهو يفحص ليس فقط الأسماء والأسعار، بل يفحص **توفر المواعيد الفعلية في جدول الحجوزات** ويستبعد الوحدات المحجوزة تلقائياً!

`GET` أو `POST` `/search`

أهم معايير الفلترة (Query Parameters):

المعامل (Parameter)	النوع	الوصف والأهمية
`city`	String (ID)	معرف المدينة (مهم لحصر النتائج في نفس المحافظة/الساحل)
`category`	String (ID/Slug)	تصنيف العقار: فندق، يخت، شاليه، جولف كار...
`regions`	String (IDs)	معرفات المناطق مفصولة بفاصلة مثل: `reg1,reg2`
`guests`	Number	عدد النزلاء/الركاب (يفحص سعة الوحدة `capacity`)
`startDate` / `endDate`	Date (YYYY-MM-DD)	تاريخ الوصول والمغادرة (يتحقق من خلو الوحدة من الحجوزات!)
`minPrice` / `maxPrice`	Number	نطاق السعر المطلوب
`unitType`	String (ID)	نوع الوحدة (ستوديو، فيلا، كابينة...)
`keyword`	String	بحث نصي بالاسم أو الوصف
`radius` , `lng` , `lat`	Floats	بحث جغرافي حول موقع المستخدم أو نقطة معينة بالكيلومتر

الفلاتر الديناميكية (Dynamic Attributes):

إذا أراد التطبيق بناء صفحة فلاتر ديناميكية حسب نوع الوحدة (مثلاً: حمام سباحة، يرى البحر، عدد الغرف):

`GET` `/unit-types/:id/attributes`

يعيد قائمة بالحقول والخصائص الممكنة ليقوم الفلاتر برسم Checkboxes و Sliders ديناميكية.

الخطوة الثامنة — شاشة تفاصيل العقار (Advertisement Details Screen)

عرض مواصفات الوحدة ومسارات الأنواع المختلفة

عند فتح صفحة أي إعلان، يتم جلب تفاصيله الكاملة:

`GET` `/advertisements/:id` عام

بناء واجهة الفلاتر حسب نوع العقار (Type Switch):

🏨 الحالة 1: الإعلان فندق (isHotel == true)

الفنادق لا تحجز مباشرة كوحدة واحدة، بل تحتوي على غرف (Rooms). يجب على الفلاتر عرض زر "اختر الغرفة":

- جلب الغرف المتاحة بالتواريخ والنزلاء:

`GET`

```
/advertisements/:id/available-rooms?
startDate= ... &endDate= ... &guests=2
```

- جلب كل غرف الفندق:

`/advertisements/:id/rooms`

⛵ الحاله 2: يخت أو عربة جولف (Yacht / Golf Car)

يتم تأجيرها بالساعة أو باليوم الكامل. يجب على الفلاتر عرض DatePicker وعند اختيار يوم، جلب السلوتات المحجوزة:

- جلب جدول مواعيد اليوم الشاغرة والمحجوزة:

`GET`

```
/bookings/schedule/:advertisementId?
date=2026-11-20
```

- يعيد السلوتات المحجوزة بالساعات وخيار اليوم الكامل.

الخطوة التاسعة والعاشرة — فحص التوفر، حاسبة السعر، وتأكيد الحجز

Universal Booking Pipeline (الحجز الموحد)

قبل أن يدفع المستخدم أو يضغط "تأكيد الحجز"، يجب حساب السعر النهائي والتأكد من خلو الوحدة من التعارضات (Conflict Check).

1. تقدير السعر وفحص التوفر (Estimate / Calculate Price):

`POST` `/bookings/calculate-price` عام

نماذج الـ Payload حسب نوع العقار:

أ) للشاليهات والفلل والفنادق (بالليالي / الأيام):

```
{
  "advertisement": "660000000000000000000010",
  "room": "660000000000000000000020", // اختياري في الفنادق فقط
  "startDate": "2026-11-01",
  "endDate": "2026-11-05"
}
```

ب) لليخوت وعربات الجولف (بالساعات):

```
{
  "advertisement": "660000000000000000000030",
  "date": "2026-11-20",
  "startTime": "14:00",
  "durationHours": 3,
  "rentalType": "hourly"
}
```

ج) لليخوت وعربات الجولف (يوم كامل Full Day):

```
{
  "advertisement": "660000000000000000000030",
  "date": "2026-11-20",
  "rentalType": "full_day"
}
```

الرد النموذجي لحاسبة السعر:

```
{
  "status": "success",
  "data": {
    "available": true,          // هل الموعد شاغر؟
    "duration": 4,             // ليالي أو 3 ساعات 4
    "pricingUnit": "per_night", // أو "per_hour" أو "per_day"
    "unitPrice": 5000,
    "totalPrice": 20000,
    "currency": "EGP",
    "minDuration": 1
  }
}
```

2. إنشاء وتأكيد الحجز النهائي (Create Booking):

إذا كانت `available == true` ووافق العميل على السعر وتفاصيل النزلاء:

`POST` `/bookings` `يحتاج توكن`

الـ Body المرسل:

```
{
  "advertisement": "660000000000000000000010",
  "room": "660000000000000000000020", // إن وجد
  "startDate": "2026-11-01",
  "endDate": "2026-11-05",
  "guestDetails": {
    "fullName": "أحمد محمود",
    "phone": "+201012345678",
    "adults": 2,
    "children": 1
  },
  "paymentMethod": "cash", // "cash", "credit_card", "wallet"
```

```
  "notes": "يرجى توفير سرير إضافي للأطفال"
}
```

يعود السيرفر بـ `201 Created` مع كائن الحجز ورقم المرجع، وحالته المبدئية `pending` .

الخطوات 11 و 12 و 13 — حجوزاتي، المفضلة، وإدارة الحساب

متابعة الحجوزات، قائمة الرغبات، والبروفايل

1. شاشة حجوزاتي (My Bookings Tab):

- عرض قائمة الحجوزات: `GET` `/bookings?status=pending&page=1&limit=10`
- عرض تفاصيل حجز: `GET` `/bookings/:id`
- إلغاء حجز: `PATCH` `/bookings/:id/cancel` ➡️ يتم تحرير الموعد في جدول الحجوزات تلقائياً.

2. شاشة المفضلة (Wishlist):

- جلب كل العناصر المفضلة: `GET` `/wishlist`
- **زر القلب الذكي (One-Click Toggle):**
  `POST` `/wishlist/toggle` (Body: `{"advertisementId": "..."}`)
  أو `POST` `/advertisements/:id/wishlist/toggle`
  يقوم السيرفر بالإضافة إن لم يكن موجوداً، أو الحذف إن كان موجوداً، ويعيد لك الحالة الجديدة!
- تفريغ المفضلة بالكامل: `DELETE` `/wishlist/clear`

3. شاشة الحساب والإعدادات (Profile Screen):

- بيانات حسابي: `GET` `/auth/me`
- تحديث الاسم ورقم الهاتف: `PATCH` `/auth/update-profile` (Body: `{"name": "...", "phone": "..."}`)
- تغيير كلمة المرور: `PATCH` `/auth/change-password`
- تسجيل الخروج: `POST` `/auth/logout` (قم بحذف التوكن محلياً)
- حذف الحساب نهائياً: `DELETE` `/auth/delete-account`

4. القاموس الفني الكامل لجميع الـ Endpoints (100% APIs)

جدول شامل ومفهرس يضم كل نقطة نهاية (Endpoint) معرفة في كود المشروع:

1. واجهات المصادقة والحساب (Auth APIs - /api/v1/auth)

Method	Endpoint	Auth؟	الوصف وبيانات الـ Body
POST	`/auth/signup`	NO	إنشاء حساب مستخدم جديد (name, email, phone, password, confirmPassword)
POST	`/auth/login`	NO	تسجيل الدخول بالبريد وكلمة المرور (email, password)
POST	`/auth/google`	NO	تسجيل الدخول عبر توكن Firebase Google (idToken)
GET	`/auth/verify-email/:token`	NO	تأكيد البريد الإلكتروني برمز التفعيل
POST	`/auth/resend-verification`	NO	إعادة إرسال رسالة تفعيل البريد الإلكتروني (email)
POST	`/auth/forgot-password`	NO	طلب كود OTP لإعادة ضبط كلمة المرور (email)
POST	`/auth/verify-reset-otp`	NO	التحقق من كود الـ OTP (email, otp) والحصول على resetToken
PATCH	`/auth/reset-password/:token`	NO	تعيين كلمة مرور جديدة (password, confirmPassword)
GET	`/auth/me`	YES	جلب بيانات الملف الشخصي للمستخدم الحالي وصلاحياته
PATCH	`/auth/update-profile`	YES	تحديث بيانات البروفايل (name, phone)
PATCH	`/auth/change-password`	YES	تغيير كلمة المرور (currentPassword, newPassword, confirmPassword)
POST	`/auth/select-city`	YES	تحديد وحفظ المدينة المفضلة للمستخدم (cityId)
POST	`/auth/select-region`	YES	تحديد وحفظ المنطقة المفضلة للمستخدم (regionId)

Method	Endpoint	Auth؟	الوصف وبيانات الـ Body
POST	/auth/logout	YES	تسجيل الخروج وإنهاء الجلسة
DELETE	/auth/delete-account	YES	حذف حساب المستخدم وجميع بياناته نهائياً

2. واجهات المدن والمناطق (Cities & Regions)

Method	Endpoint	Auth؟	الوصف
GET	/cities	NO	قائمة المدن المتاحة وعدد مناطق كل مدينة (active=true)
GET	/cities/:id	NO	تفاصيل مدينة محددة وقائمة مناطقها التابعة
GET	/cities/:id/regions	NO	جلب مناطق مدينة معينة فقط (سيدي عبد الرحمن، مراسي...)
GET	/cities/:id/data	NO	**شاشة المدينة الشاملة:** تعيد المدينة + المناطق + البانرات + أبرز الإعلانات
GET	/regions	NO	قائمة المناطق (يمكن فلترتها بـ ?city=cityId)
GET	/regions/:id	NO	تفاصيل منطقة معينة وحدودها الجغرافية
GET	/regions/:id/data	NO	**شاشة المنطقة الشاملة:** بانرات المنطقة + الشاليهات + كل عقاراتها

3. واجهات الإعلانات والبحث والغرف (Ads, Search & Rooms)

Method	Endpoint	Auth؟	الوصف
GET	/advertisements	NO	قائمة الإعلانات مع فلترة بـ city, category, unitType, page, limit
GET	/advertisements/:id	NO	التفاصيل الكاملة لإعلان (المواصفات، الأسعار، الصور، المالك)
GET	/advertisements/:id/rooms	NO	قائمة الغرف التابعة لفندق معين
GET	/advertisements/:id/available-rooms	NO	الغرف المتاحة بالفندق بالتواريخ والنزلاء (?startDate=&endDate=&guests=)

Method	Endpoint	?Auth	الوصف
GET / POST	`/search`	NO	محرك البحث والفلترة الذكي (تواريخ، أسعار، نزلاء، موقع، تصنيفات)
GET	`/rooms/:id`	NO	تفاصيل غرفة معينة داخل فندق (السرائر، السعة، السعر)
GET	`/unit-types`	NO	قائمة أنواع الوحدات (فيلا، كابينة، شقة، ستوديو...)
GET	`/unit-types/:id/attributes`	NO	الخصائص والحقول الديناميكية لنوع الوحدة لبناء فلاتر متقدمة
GET	`/banners`	NO	البانرات الإعلانية (?city=®ion=&position=top|center|bottom)
GET	`/categories`	NO	التصنيفات الرئيسية (فنادق، شاليهات، يخوت، عربات جولف)
GET	`/onboarding`	NO	شرائح التعريف والشرح لتطبيق الموبايل

4. واجهات الحجوزات والمفضلة (Bookings & Wishlist)

Method	Endpoint	?Auth	الوصف
GET	`/bookings/check-availability/:id`	NO	فحص توفر عقار أو غرفة محددة في تواريخ معينة
GET	`/bookings/schedule/:id`	NO	سلوتات ومواعيد اليخت أو الجولف كار بيوم محدد (?date=YYYY-MM-DD)
POST	`/bookings/calculate-price`	NO	**حاسبة السعر الموحدة:** تفحص التوفر وتحسب السعر الدقيق قبل الحجز
POST	`/bookings`	YES	**إنشاء حجز جديد:** (فندق، شاليه، يخت، جولف كار)
GET	`/bookings`	YES	عرض قائمة حجوزات المستخدم الحالي (?status=pending|confirmed...)

Method	Endpoint	Auth؟	الوصف
GET	/bookings/:id	YES	عرض التفاصيل الكاملة لحجز معين وإيصاله المالي
PATCH	/bookings/:id/cancel	YES	إلغاء حجز قائم وتحرير الموعد في جدول المواعيد
GET	/wishlist	YES	عرض قائمة الإعلانات المضافة للمفضلة للمستخدم الحالي
POST	/wishlist/toggle	YES	تبديل حالة المفضلة بنقرة واحدة (Body: {"advertisementId": "..."})
POST	/advertisements/:id/wishlist/toggle	YES	تبديل حالة المفضلة من صفحة تفاصيل الإعلان مباشرة
DELETE	/wishlist/clear	YES	مسح كافة العناصر من قائمة المفضلة

5. ملخص واجهات الإدارة (Admin APIs - /api/v1/admin/*)

هذه الواجهات مخصصة للوحة التحكم والأدمن والمالك (Owner):

الوحدة	المسارات (Endpoints)	الصلاحيات
Admin Auth	`POST /admin/auth/login` , `POST /admin/auth/google` , `GET /admin/auth/me`	Admin / Super Admin
Users Management	`GET /admin/users` , `GET /stats` , `POST /` , `PATCH /:id/toggle-status`	Super Admin
Advertisements Admin	CRUD كامل، `POST /:id/owner` (تعيين مالك)، `POST /:id/images` (رفع حتى 10 صور)، `PATCH /set-main-image`	Admin / Owner
Bookings Admin	CRUD كامل على جميع الحجوزات، تغيير الحالة (confirm, complete, cancel)، وفحص التوفر	Admin
Cities, Regions & Banners	إضافة وتعديل وحذف المدن والمناطق ورسم إحداثيات الـ Polygons ورفع الصور	Admin
Unit Types & Attributes	إدارة أنواع الوحدات وإضافة وحذف الخصائص الديناميكية (Attributes)	Admin

الوحدة	المسارات (Endpoints)	الصلاحيات
Onboarding Admin	إضافة وتعديل وحذف شرائح التعريف وترتيبها `PATCH /reorder/bulk`	Admin

5. نصائح وأكواد مساعدة جاهزة لـ Flutter

أكواد معمارية جاهزة تمكن مطور الفلاتر من ربط المشروع باحترافية في دقائق معدودة:

1. إعداد عميل Dio والـ Interceptors

يتولى إرفاق الـ Token واللغة المختارة ومعالجة أخطاء 401 تلقائياً:

```
import 'package:dio/dio.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

class ApiClient {
  static const String baseUrl = 'http://10.0.2.2:3000/api/v1'; // أو سيرفر الإنتاج
  late final Dio dio;
  final FlutterSecureStorage _storage = const FlutterSecureStorage();

  ApiClient() {
    dio = Dio(
      BaseOptions(
        baseUrl: baseUrl,
        connectTimeout: const Duration(seconds: 15),
        receiveTimeout: const Duration(seconds: 15),
        headers: {
          'Content-Type': 'application/json',
          'Accept': 'application/json',
          'Accept-Language': 'ar', // يدعم 'ar' أو 'en'
        },
      ),
    );

    dio.interceptors.add(
      InterceptorsWrapper(
        onRequest: (options, handler) async {
          final token = await _storage.read(key: 'jwt_token');
          if (token != null && token.isNotEmpty) {
            options.headers['Authorization'] = 'Bearer $token';
          }
          return handler.next(options);
        },
        onError: (DioException error, handler) async {
          if (error.response?.statusCode == 401) {
            // التوكن منتهي أو غير صالح
            await _storage.delete(key: 'jwt_token');
            // يمكنك هنا إطلاق حدث لإعادة التوجيه لصفحة تسجيل الدخول
          }
          return handler.next(error);
        },
      ),
    );
```

```
  }
}
```

2. كود Generic ApiResponse Model

نموذج Dart موحد يتعامل مع كل ردود السيرفر تلقائياً:

```dart
class ApiResponse<T> {
  final String status;
  final String? message;
  final T? data;
  final int? results;
  final int? total;
  final int? page;
  final int? totalPages;
  final String? token;

  ApiResponse({
    required this.status,
    this.message,
    this.data,
    this.results,
    this.total,
    this.page,
    this.totalPages,
    this.token,
  });

  bool get isSuccess => status == 'success';

  factory ApiResponse.fromJson(
    Map<String, dynamic> json,
    T Function(dynamic dataJson)? fromDataJson,
  ) {
    return ApiResponse<T>(
      status: json['status'] ?? 'fail',
      message: json['message'],
      data: (json['data'] != null && fromDataJson != null)
          ? fromDataJson(json['data'])
          : json['data'] as T?,
      results: json['results'],
      total: json['total'],
      page: json['page'],
      totalPages: json['totalPages'],
      token: json['token'],
    );
  }
}
```

3. دالة معالجة روابط الصور (Image URL Resolver)

```
String resolveImageUrl(String? rawUrl) {
  if (rawUrl == null || rawUrl.isEmpty) return 'assets/images/placeholder.png';
  if (rawUrl.startsWith('http://') || rawUrl.startsWith('https://')) {
    return rawUrl;
  }
  const String serverDomain = 'http://10.0.2.2:3000';
  final cleanPath = rawUrl.startsWith('/') ? rawUrl : '/$rawUrl';
  // إذا كان المسار يحتوي بالفعل على uploads
  if (cleanPath.startsWith('/uploads/')) {
    return '$serverDomain$cleanPath';
  }
  return '$serverDomain/uploads$cleanPath';
}
```

✨ ملخص تنفيذي لمطور فلاتر (Quick Cheat Sheet)

1. **أول شاشة:** نادِ `GET /cities` واحفظ الـ `cityId` في State إداري عام.
2. **الصفحة الرئيسية:** استخدم دائماً `GET /cities/:id/data?position=top` لجلب كل محتويات الصفحة بطلب واحد.
3. **البحث:** استخدم `GET /search` مع تمرير `city` و `startDate` و `endDate` لاستبعاد الوحدات المحجوزة.
4. **قبل الحجز:** استدعِ دائماً `POST /bookings/calculate-price` للتأكد من التوفر وعرض السعر الإجمالي للمستخدم.
5. **عند الحجز:** أرسل `POST /bookings` مع الـ Bearer Token وتفاصيل الضيف.
6. **زر القلب:** استخدم `POST /wishlist/toggle` لإضافة أو إزالة الوحدة بلمسة واحدة.